Web Services

HTTP, Request Headers, RESTful Web Services, Swagger

SoftUni Team

Technical Trainers

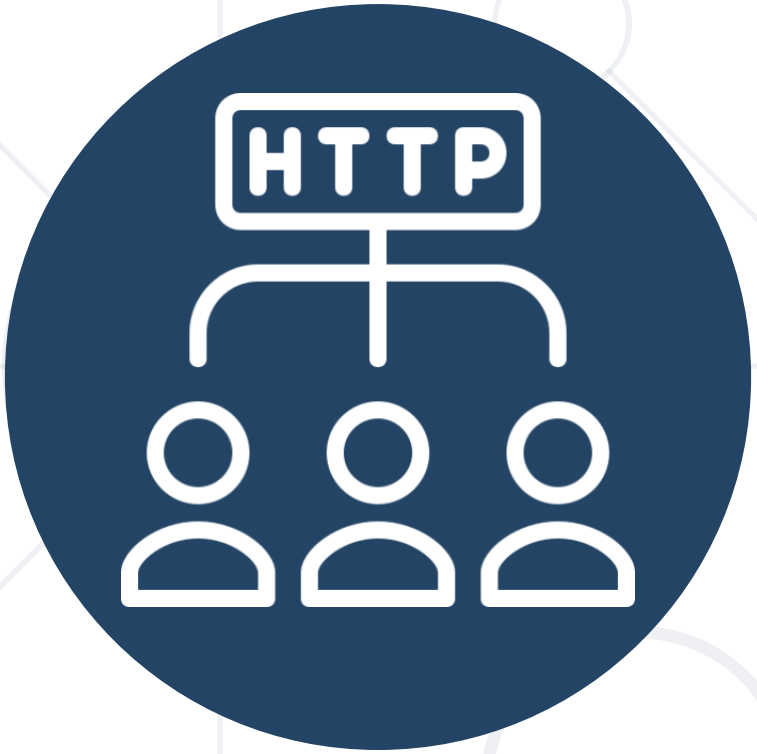
Software University

SoftUni

Software University

https://softuni.bg

Table of Contents

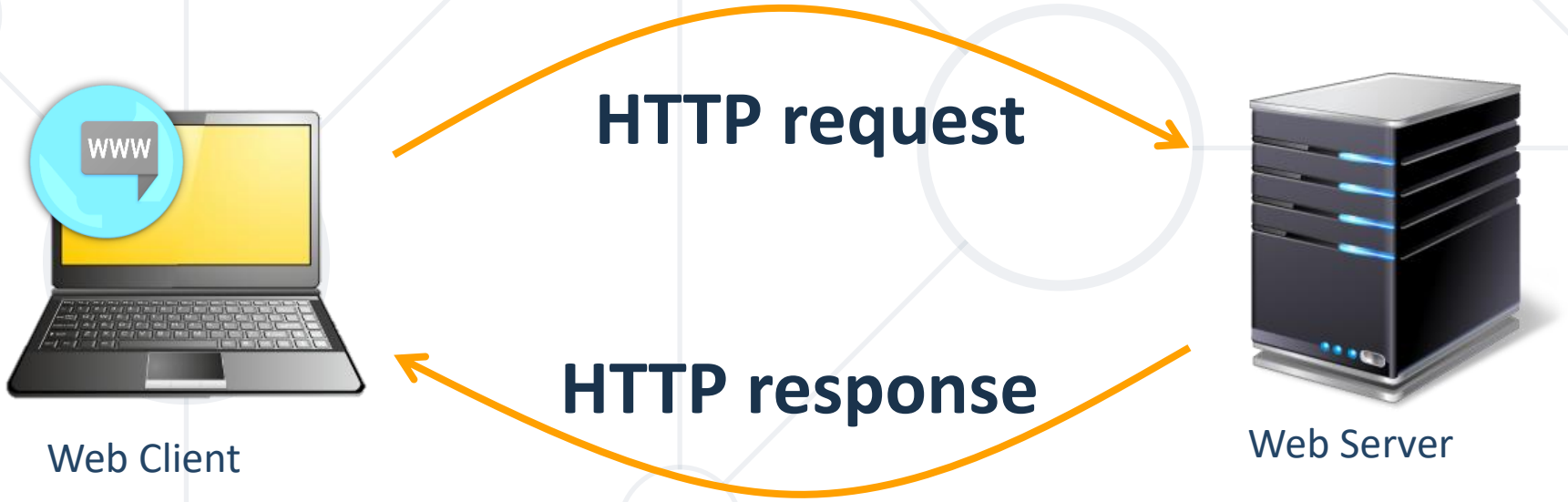


HTTP Overview

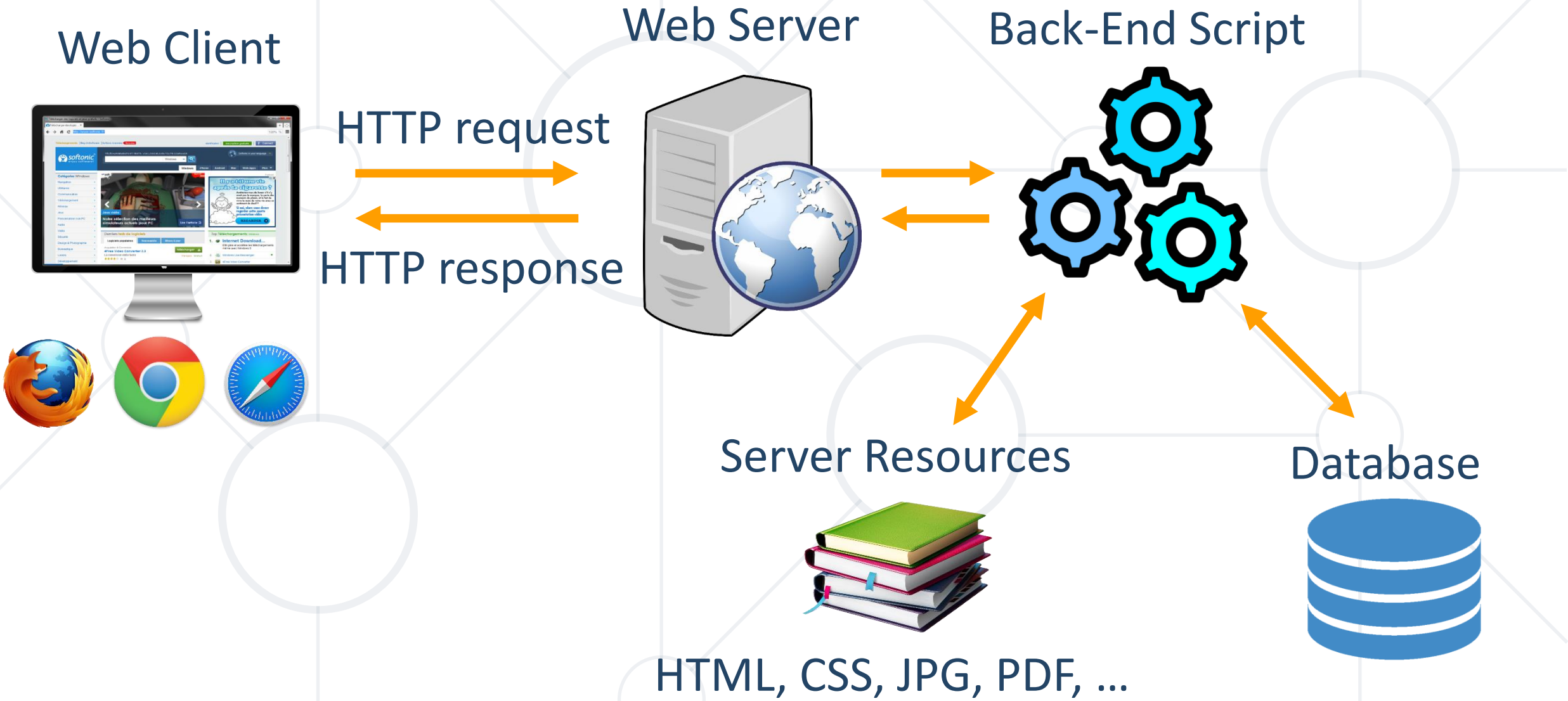
Hypertext Transfer Protocol

HTTP Basics

- HTTP (**H**yper **T**ext **T**ransfer **P**rotocol)
  - Text-based client-server protocol for the Internet
  - For transferring Web resources (HTML files, images, styles, etc.)
  - Request-response based



Web Server Work Model



HTTP Request Methods

- **HTTP request methods** specify the desired **action** to be performed on the requested resource (identified by URL)

Method	Description
GET 	Retrieve a resource
POST 	Create / store a resource
PUT 	Update (replace) a resource
DELETE 	Delete (remove) a resource
PATCH 	Update resource partially (modify)
HEAD 	Retrieve the resource's headers

CRUD == the four main functions of persistent storage

Other Methods
CONNECT
OPTIONS
TRACE

HTTP Response Status Codes

Software University

Status Code	Action	Description	
200	OK	Successfully retrieved resource	Success
201	Created	A new resource was created	
204	No Content	Request has nothing to return	
301 / 302	Moved	Moved to another location (redirect)	Redirect
400	Bad Request	Invalid request / syntax error	Error
401 / 403	Unauthorized	Authentication failed / access denied	
404	Not Found	Invalid resource requested	
409	Conflict	Conflict detected, e.g. duplicated email	
500 / 503	Server Error	Internal server error / service unavailable	

Content-Type and Disposition

- The **Content-Type** / **Content-Disposition** headers specify how the HTTP request / response body should be processed

JSON-encoded data

```
Content-Type: application/json
```

UTF-8 encoded HTML page.
Will be shown in the browser

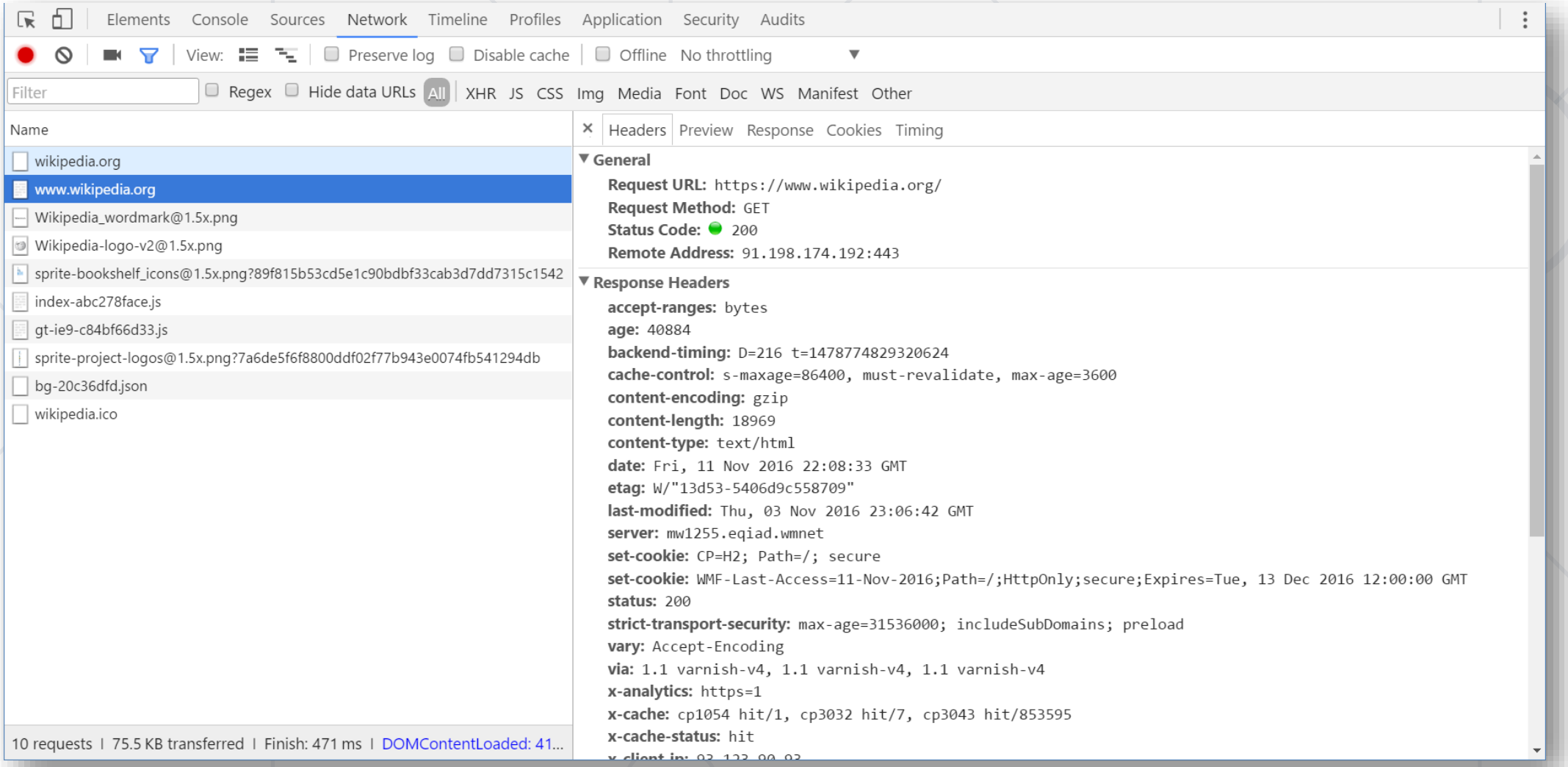
```
Content-Type: text/html; charset=utf-8
```

```
Content-Type: application/pdf
Content-Disposition: attachment;
filename="Financial-Report-April-2016.pdf"
```

This will download a PDF file named
Financial-Report-April-2016.pdf

Browser Developer Tools

Software University



Introduction to RESTful Services

Mapping CRUD Operations

What is an API?

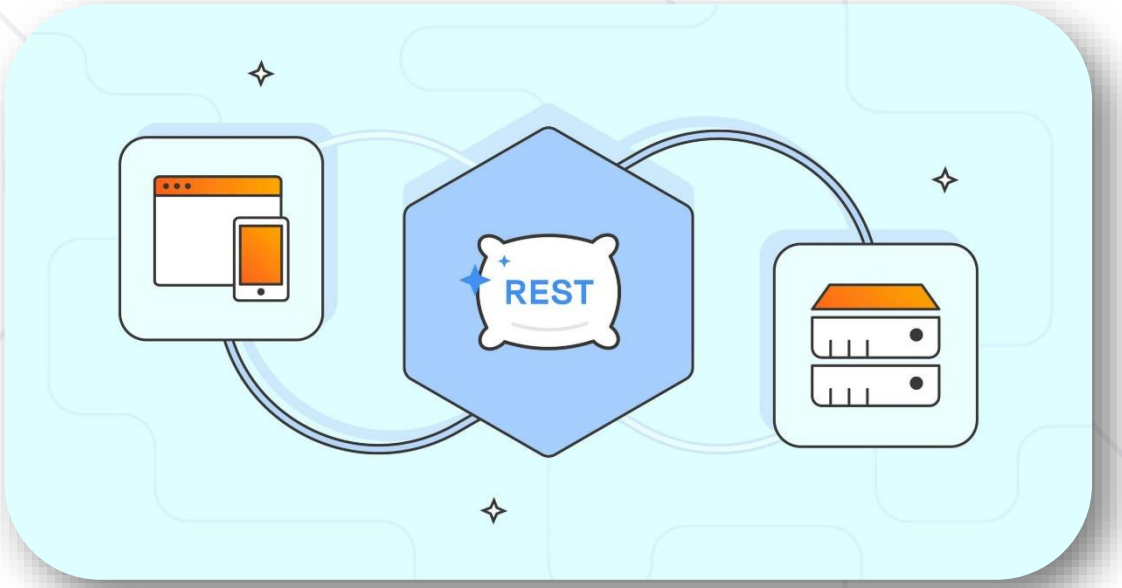
- **A**pplication **P**rogramming **I**nterface
  - **API**s – Mechanisms that enable **two software components** to **communicate with each other** using a set of definitions and **protocols**
- **API**s in everyday life
  - Social media bots
  - Third-party login
  - E-commerce transactions
  - Weather apps

What is RESTful API?

- An **application programming interface** that follows the principles of **REpresentational State Transfer**
- **REST** - a **set of guidelines** for designing web services that are **scalable**, **uniform**, and **stateless**
- Allows clients to **access and manipulate resources** on a server using standard **HTTP methods** – **GET**, **POST**, **PUT**, and **DELETE**
- **Resource** - anything that **has a unique identifier**, such as a user, a product, or a post

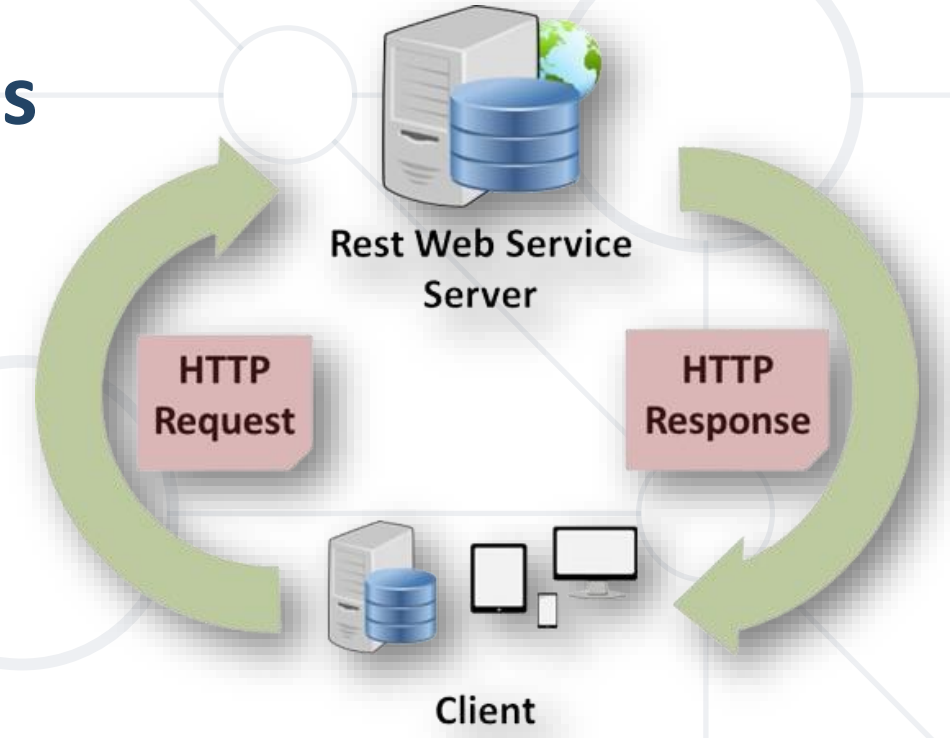
REST - Communication Standard

- **REST** - the most common communication standard between computers over Internet
- The **common API standard** used by most mobile and web **applications to talk to the servers is called REST**



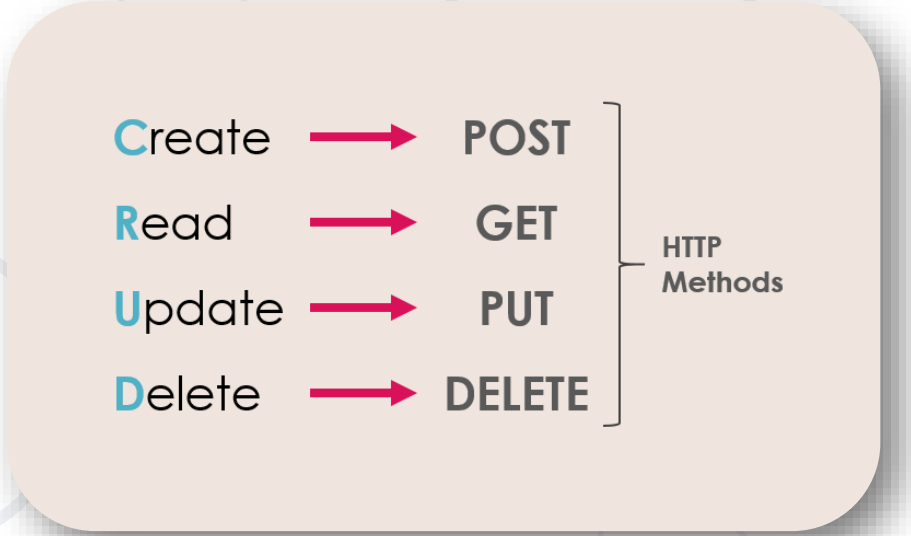
REST Architecture

- In **REST architecture**, a **REST Server** simply **provides access to resources** and **REST client** **accesses and modifies** the resources
- Each resource is identified by **URIs / global IDs**
- REST uses various **formats to represent a resource** like text, JSON, XML…



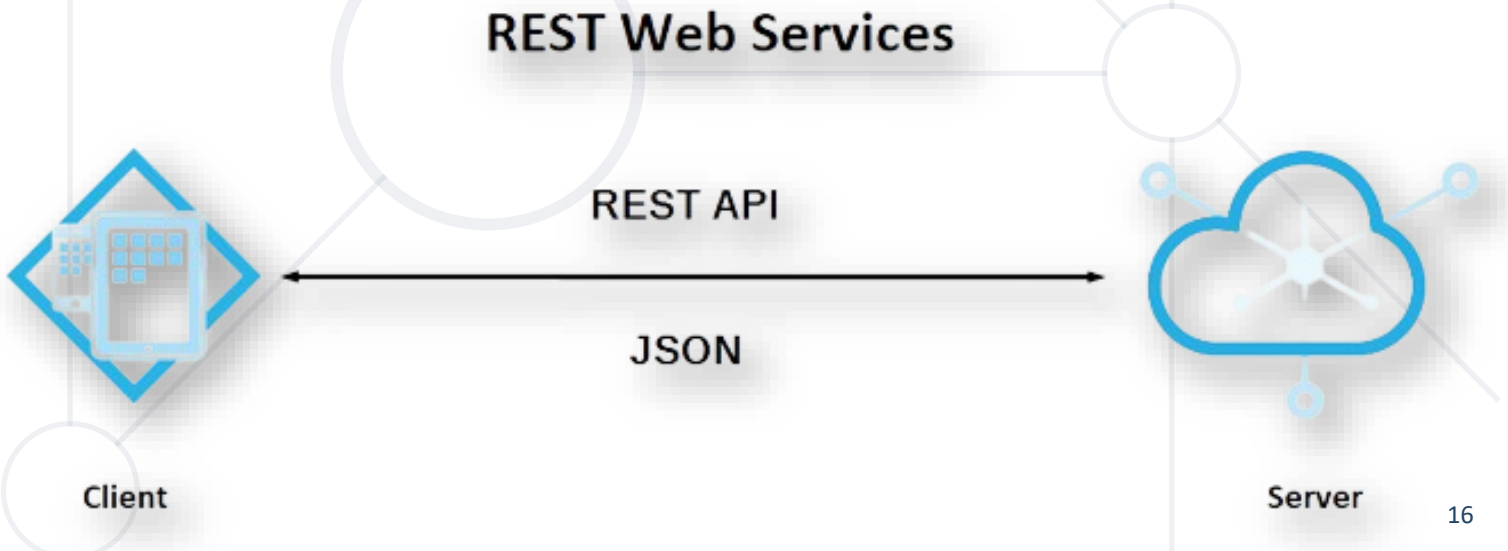
HTTP Methods –> CRUD Operations

- The following **HTTP methods** are commonly used in **REST-based architecture**:
  - **POST** – Used to **create** a new resource
  - **GET** – Provides a **read-only access** to a resource
  - **PUT** – Used to **update** an existing resource or create a new resource
  - **DELETE** – Used to **remove** a resource



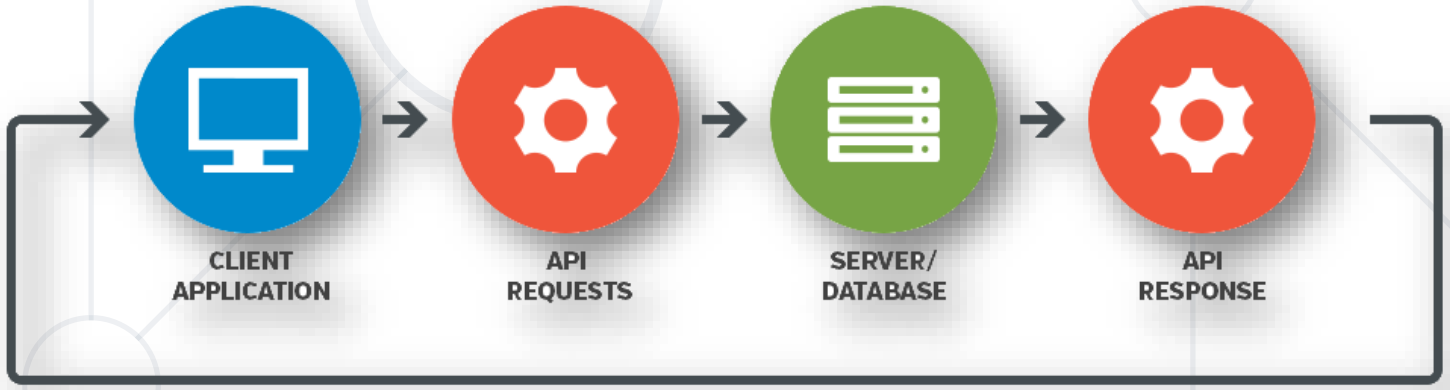
REST Web Services

- **REST Web Service**
  - A **lightweight**, **maintainable**, and **scalable** service
  - Built on the **REST architecture**
- **Expose API** from an application **to the calling client** in a
  - Secure
  - Uniform
  - Stateless manner



Web Services and APIs

- **Web services** expose **back-end APIs** over the **network**
  - May use different **protocols** and **data formats**: HTTP, REST, GraphQL, gRPC, SOAP, JSON-RPC, JSON, BSON, XML, YML, …
- **Web services** are hosted on a Web server (HTTP server)
  - Provide a set of functions, invokable from the Web (Web API)
- **RESTful APIs** is the most popular Web service standard



Authentication and Authorization

RESTful APIs

Authentication

- The process of **verifying the identity of a user or a system**
- This process **often** involves checking whether a **username and password provided are correct**
- **In RESTful services**, authentication can be achieved through various methods, including
  - Basic authentication (using a base64 encoded **username and password**)
  - **Tokens** such as JSON Web Tokens (JWTs)
  - **HMAC** (hash-based message **authentication codes**)

Authorization

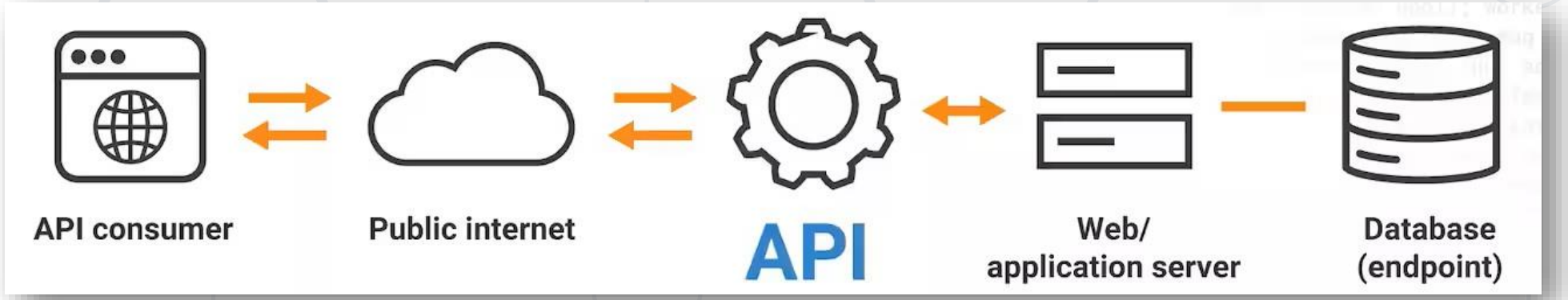
- **Authorization** - granting an **authenticated user permission** to access different resources or perform specific actions
- **Once a user is authenticated**, the system must check **what resources the user is allowed to access** or **what actions** they are **permitted to perform**
- **It's important to note that both authentication and authorization mechanisms must be protected with HTTPS to prevent man-in-the-middle attacks and to ensure that sensitive information is transmitted securely**

Authentication vs Authorization

- **Authentication**
  - Determines **whether users are** who they claim to be
  - Generally, transmits info through an **ID Token**
  - Usually done before authorization
- **Authorization**
  - Determines **what users can and cannot access**
  - Generally, transmits info through an **Access Token**
  - Usually done after successful authentication

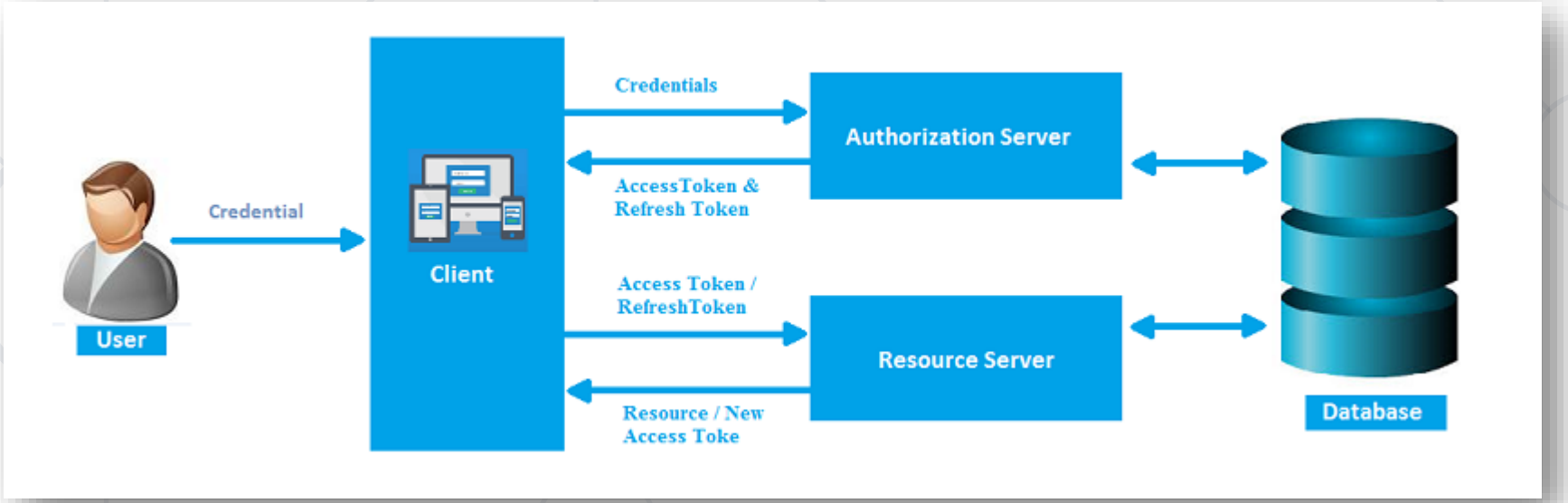
REST API Authentication

- **REST APIs** have become approach for modern **web** and **mobile application platforms**
- They **separate data** and **presentation layers**, allowing systems to scale in size and feature sophistication over time
- As **data moves across boundaries**, **security** becomes a **key concern** for REST APIs containing **sensitive information**



REST API Authentication

- One of the most straightforward ways to **secure these APIs** is to implement **authentication mechanisms** that control their exposure
- Mainly through **user credentials** and **encrypted access codes**

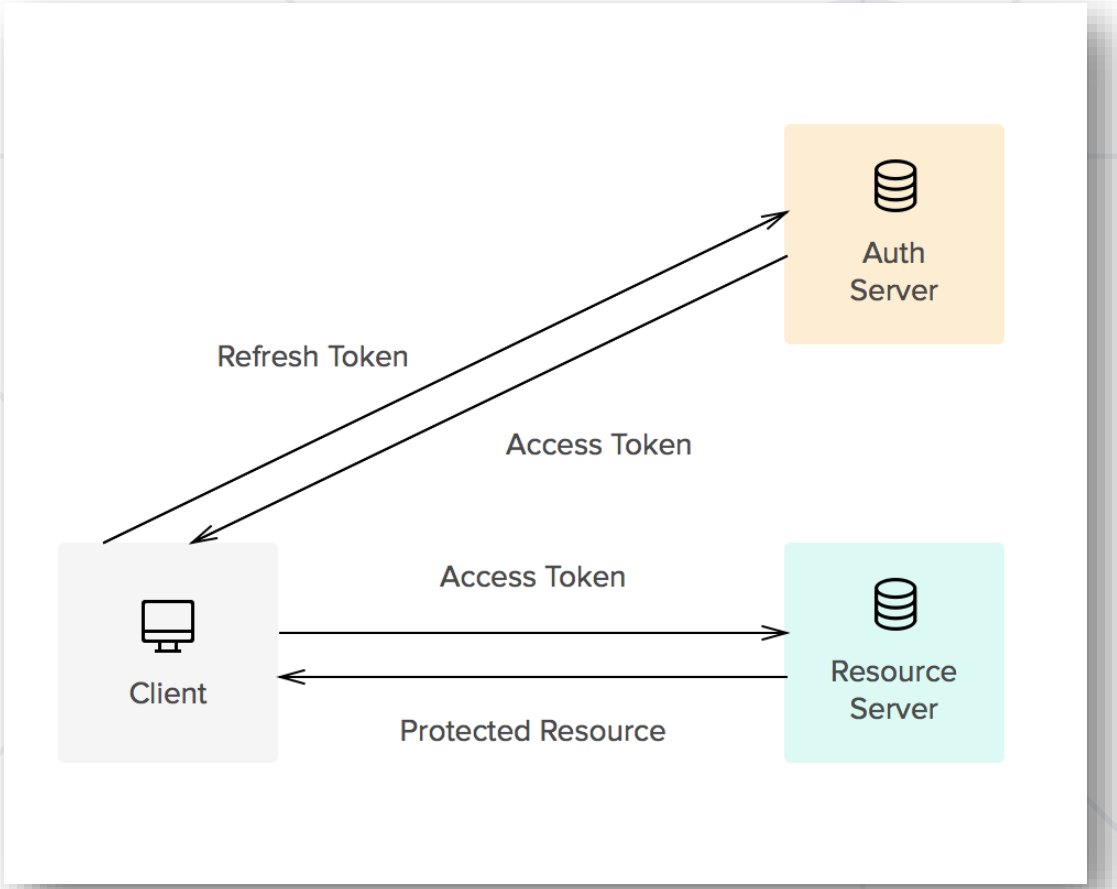


Token Authentication

- Token authentication is also known as **Bearer Authentication**
- To use it, you just specify Authorization:
  - **Bearer <token>**
- Token is a string that **represents** the **user's identity** and **permissions**
  - If you **have (bear) the token**, you can get the appropriate **access to the API**

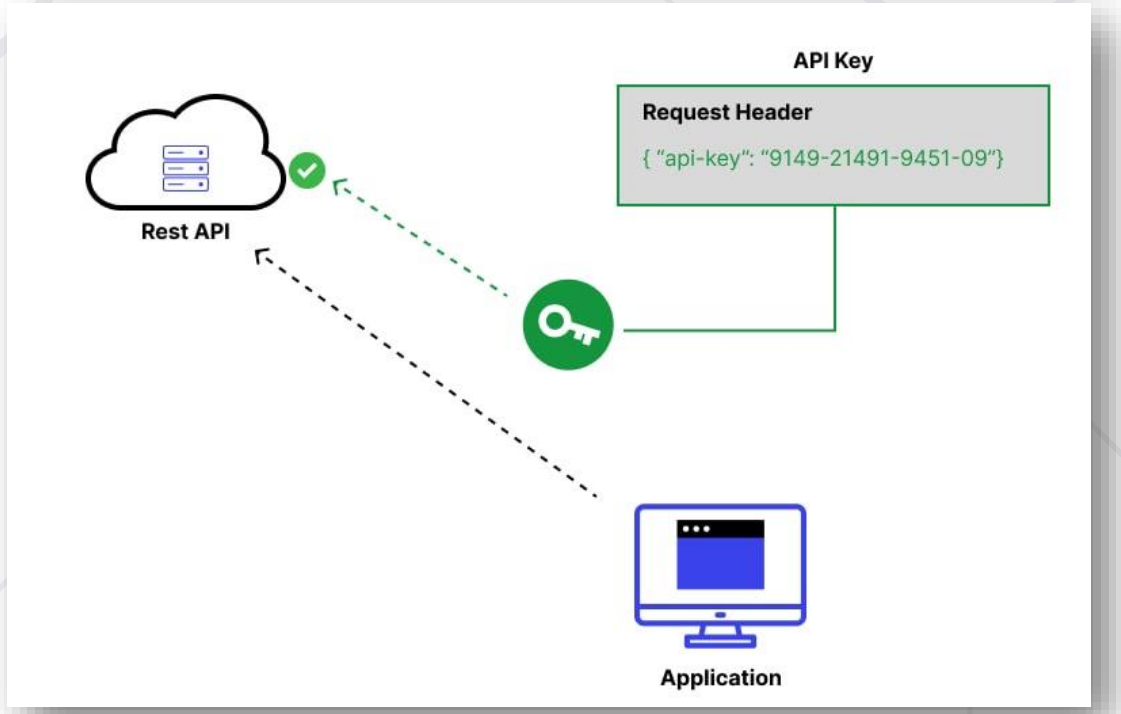
Bearer Authentication

- **Bearer Authentication** is a method of **sending a token with HTTP requests** to authenticate
- The token is a string, often in **JWT (JSON Web Token)** format, that the server can use to verify the request's authenticity and integrity



REST API Authorization

- After you prove the **user's identity**, you can check which data that user is **allowed to access**
- Authorization ensures that the **user is authorized** to **view** or **edit** a specific **set of data**



Swagger

API Documentation and Testing Tool

What is Swagger UI?

- Swagger is an **open-source framework** that helps developers **design**, **build**, **document**, and **consume RESTful web services**
- It simplifies **API development** by offering a **standardized approach to documenting APIs**
- Provides clear guidance for developers on **how to interact with an API**
- Reduces confusion and errors during integration

Benefits

- It creates **consistent documentation** across teams and projects
- Developers can try **API calls** directly in the browser with Swagger UI
- With tools like **Swagger Codegen**, you can automatically generate client SDKs and server stubs
- Validates **API requests** and **responses** against the API contract
- Teams can **work together** on API design and documentation

Swagger UI Example

Software University

Swagger Editor Supported by SMARTBEAR | File | Edit | Insert | Generate Server | Generate Client | About | Try our new Editor ↗

```yaml
paths:
  /pet:
    put:
      tags:
        - pet
      summary: Update an existing pet
      description: Update an existing pet by Id
      operationId: updatePet
      requestBody:
        description: Update an existent pet in the store
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Pet'
          application/xml:
            schema:
              $ref: '#/components/schemas/Pet'
          application/x-www-form-urlencoded:
            schema:
              $ref: '#/components/schemas/Pet'
        required: true
      responses:
        '200':
          description: Successful operation
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Pet'
            application/xml:
              schema:
                $ref: '#/components/schemas/Pet'
        '400':
          description: Invalid ID supplied
        '404':
          description: Pet not found
        '422':
          description: Validation exception
      security:
```

pet Everything about your Pets — Find out more

PUT /pet Update an existing pet

POST /pet Add a new pet to the store

Add a new pet to the store

Parameters | Try it out

No parameters

Request body required — application/json

Create a new pet in the store

Example Value | Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
```

Summary

- Hypertext Transfer Protocol
- HTTP Response Status Codes
- RESTful Services – Introduction
- RESTful APIs
- Authentication and Authorization in REST API

Questions?

SoftUni

Software University

SoftUni AI

SoftUni Digital

SoftUni Creative

SoftUni Foundation

СофтУни БУДИТЕЛ

Finance Academy

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg, about.softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://softuni.org
- © Software University – https://softuni.bg